

Practicing R with swirl

Atividade semanal

Bruna Marques Moreno

Na atividade de realizar o curso *R Programming: The basics of programming in R* eu realizei o módulo *R Programming Basic Building Blocks*. Neste, eu pude rever comandos básicos novamente, como criar vetores:

```
novo_vetor <- c()  
  
# É possível acrescentar números, letras e palavras num vetor
```

entender como qualquer ação afeta os valores dentro dos vetores. Sabia manipular vetores, mas não este detalhe simples de que de vetores de tamanhos distintos realizam o comando até o ponto em que há o mesmo número de elementos entre os vetores.

```
# To see another example of how this vector 'recycling' works, try  
# adding c(1, 2, 3, 4) and c(0, 10). Don't worry about saving the  
# result in a new variable.  
  
c(1,2,3,4) + c(0,10)
```

```
[1]  1 12  3 14
```

Pessoalmente, gostei da pacote, porque podemos aprender sem necessariamente ter uma plataforma externa ao R que reduz as chances de concluir um curso. E ele tem uma grande diversidade de temas. No entanto, tive dificuldade de sair do curso concluído e começar outro. Busquei alternativas na internet, mas não encontrei. Testei o comando `bye()` para ver se voltava para o menu inicial, mas sem sucesso. Recorri a ferramenta de IA (Google Gemini) para entender como baixar outros cursos.

```
#Para baixar novos cursos, é só pegar o nome na lista da pasta do GitHub do pacote  
  
install_course("Exploratory_Data_Analysis")
```

Assim, consegui baixar o curso *Exploratory Data Analysis* e o iniciei.

1 Saída do console

```

> library(swirl)

> swirl()

| Welcome to swirl! Please sign in. If you've been here before, use the same name
as you did then. If you are new, call yourself
| something unique.

What shall I call you? Bruna

| Would you like to continue with one of these lessons?

1: R Programming Basic Building Blocks
2: No. Let me start something new.

Seleção: 1

| If at any point you'd like more information on a particular topic related to R, you
can type help.start() at the prompt, which
| will open a menu of resources (either within RStudio or your default web browser,
depending on your setup). Alternatively, a
| simple web search often yields the answer you're looking for.

...

|=====
| In its simplest form, R can be used as an interactive calculator. Type 5 + 7 and press

> 5+7
[1] 12

| Keep up the great work!

|=====
| R simply prints the result of 12 by default. However, R is a programming language
and often the reason we use a programming
| language as opposed to a calculator is to automate some process or avoid unnecessary
repetition.

...

|=====
| In this case, we may want to use our result from above in a second calculation.
Instead of retyping 5 + 7 every time we need it,
| we can just create a new variable that stores the result.

...

```

```

|=====
| The way you assign a value to a variable in R is by using the assignment operator,
| which is just a 'less than' symbol followed
| by a 'minus' sign. It looks like this: <-
...

|=====
| Think of the assignment operator as an arrow. You are assigning the value on the
| right side of the arrow to the variable name on
| the left side of the arrow.
...

|=====
| To assign the result of 5 + 7 to a new variable called x, you type x <- 5 + 7.
| This can be read as 'x gets 5 plus 7'. Give it a
| try now.

> x <- 5+7

| That's the answer I was looking for.

|=====
| You'll notice that R did not print the result of 12 this time. When you use the
| assignment operator, R assumes that you don't
| want to see the result immediately, but rather that you intend to use the result
| for something else later on.
...

|=====
| To view the contents of the variable x, just type x and press Enter. Try it now.

> x
[1] 12

| Excellent work!

|=====
| Now, store the result of x - 3 in a new variable called y.

> y <- x - 3

| You got it!

|=====
| What is the value of y? Type y to find out.

> y
[1] 9

```

```

| That's the answer I was looking for.

|=====
| Now, let's create a small collection of numbers called a vector. Any object that
contains data is called a data structure and
| numeric vectors are the simplest type of data structure in R. In fact, even a
single number is considered a vector of length
| one.

...

|=====
| The easiest way to create a vector is with the c() function, which stands for
'concatenate' or 'combine'. To create a vector
| containing the numbers 1.1, 9, and 3.14, type c(1.1, 9, 3.14). Try it now and
store the result in a variable called z.

> z <- c(1.1, 9, 3.14)

| Keep up the great work!

|=====
| Anytime you have questions about a particular function, you can access R's
built-in help files via the `?` command. For example,
| if you want more information on the c() function, type ?c without the parentheses
that normally follow a function name. Give it
| a try.

> ?c

| Keep working like that and you'll get there!

|=====
| Type z to view its contents. Notice that there are no commas separating the
values in the output.

> z
[1] 1.10 9.00 3.14

| All that practice is paying off!

|=====
| You can combine vectors to make a new vector. Create a new vector that contains
z, 555, then z again in that order. Don't assign
| this vector to a new variable, so that we can just see the result immediately.

> c(z, 555, z)
[1] 1.10 9.00 3.14 555.00 1.10 9.00 3.14

| You are doing so well!

```

```

|=====
| Numeric vectors can be used in arithmetic expressions. Type the following to
| see what happens: z * 2 + 100.

> z * 2 + 100
[1] 102.20 118.00 106.28

| Excellent job!

|=====
| First, R multiplied each of the three elements in z by 2. Then it added 100 to
| each element to get the result you see above.

...

|=====
| Other common arithmetic operators are `+`, `-`, `/`, and `^` (where x^2 means
| 'x squared'). To take the square root, use the
| sqrt() function and to take the absolute value, use the abs() function.

...

|=====
| Take the square root of z - 1 and assign it to a new variable called my_sqrt.

> my_sqrt <- sqrt(z-1)

| That's the answer I was looking for.

|=====
| Before we view the contents of the my_sqrt variable, what do you think it contains?

1: a single number (i.e a vector of length 1)
2: a vector of length 0 (i.e. an empty vector)
3: a vector of length 3

Seleção: 3

| Excellent work!

|=====
| Print the contents of my_sqrt.

> my_sqrt
[1] 0.3162278 2.8284271 1.4628739

| That's correct!

|=====
| As you may have guessed, R first subtracted 1 from each element of z, then took

```

```

the square root of each element. This leaves you
| with a vector of the same length as the original vector z.

...

|=====
| Now, create a new variable called my_div that gets the value of z divided by my_sqrt.

> my_div <- z/my_sqrt

| Your dedication is inspiring!

|=====
| Which statement do you think is true?

1: my_div is a single number (i.e a vector of length 1)
2: my_div is undefined
3: The first element of my_div is equal to the first element of z divided by the
first element of my_sqrt, and so on...

Seleção: 3

| Keep up the great work!

|=====
| Go ahead and print the contents of my_div.

> my_div
[1] 3.478505 3.181981 2.146460

| That's the answer I was looking for.

|=====
|===== | 71%
| When given two vectors of the same length, R simply performs the specified
arithmetic operation (`+`, `-`, `*`, etc.)
| element-by-element. If the vectors are of different lengths, R 'recycles'
the shorter vector until it is the same length as the
| longer vector.

...

|=====
|===== | 74%
| When we did z * 2 + 100 in our earlier example, z was a vector of length 3, but
technically 2 and 100 are each vectors of length
| 1.

...

|=====

```

```

===== | 76%
| Behind the scenes, R is 'recycling' the 2 to make a vector of 2s and the 100
to make a vector of 100s. In other words, when you
| ask R to compute z * 2 + 100, what it really computes is this: z * c(2, 2, 2) +
c(100, 100, 100).

...

|=====
===== | 79%
| To see another example of how this vector 'recycling' works, try adding c(1, 2, 3, 4)
and c(0, 10). Don't worry about saving the
| result in a new variable.

> c(1,2,3,4) + c(0,10)
[1] 1 12 3 14

| Nice work!

|=====
===== | 82%
| If the length of the shorter vector does not divide evenly into the length of
the longer vector, R will still apply the
| 'recycling' method, but will throw a warning to let you know something fishy
might be going on.

...

|=====
===== | 84%
| Try c(1, 2, 3, 4) + c(0, 10, 100) for an example.

> c(1,2,3,4) + c(0,10,100)
[1] 1 12 103 4
Mensagem de aviso:
In c(1, 2, 3, 4) + c(0, 10, 100) :
comprimento do objeto maior não é múltiplo do comprimento do objeto menor

| That's correct!

|=====
===== | 87%
| Before concluding this lesson, I'd like to show you a couple of time-saving tricks.

...

|=====
===== | 89%
| Earlier in the lesson, you computed z * 2 + 100. Let's pretend that you made
a mistake and that you meant to add 1000 instead of
| 100. You could either re-type the expression, or...

```

```

...

|=====
|                                     | 92%
| In many programming environments, the up arrow will cycle through previous
| commands. Try hitting the up arrow on your keyboard
| until you get to this command (z * 2 + 100), then change 100 to 1000 and hit
| Enter. If the up arrow doesn't work for you, just
| type the corrected command.

>
>
> z * 2 + 1000
[1] 1002.20 1018.00 1006.28

| You got it right!

|=====
|                                     | 95%
| Finally, let's pretend you'd like to view the contents of a variable that you
| created earlier, but you can't seem to remember if
| you named it my_div or myDiv. You could try both and see what works, or...

...

|=====
|                                     | 97%
| You can type the first two letters of the variable name, then hit the Tab key
| (possibly more than once). Most programming
| environments will provide a list of variables that you've created that begin
| with 'my'. This is called auto-completion and can
| be quite handy when you have many variables in your workspace. Give it a try.
| (If auto-completion doesn't work for you, just
| type my_div and press Enter.)

> my_div
[1] 3.478505 3.181981 2.146460

| Excellent work!

|=====
| 100%
| Would you like to receive credit for completing this course on Coursera.org?

1: Yes
2: No

Seleção: 1
What is your email address? bruna.marques.moreno@usp.br
What is your assignment token? yes

```



```
Grade submission failed.
Press ESC if you want to exit this lesson and you
want to try to submit your grade at a later time.

| That's not exactly what I'm looking for. Try again.

|

1: No
2: Yes

Seleção: 2
What is your email address? bruna.marques.moreno@usp.br
What is your assignment token? yes
Grade submission failed.
Press ESC if you want to exit this lesson and you
want to try to submit your grade at a later time.

| Keep trying!

|

1: No
2: Yes

Seleção: 2
What is your email address? bruna.moreno@outlook.com.br
What is your assignment token? yes
Grade submission failed.
Press ESC if you want to exit this lesson and you
want to try to submit your grade at a later time.

| Almost! Try again.

|

1: No
2: Yes

Seleção: 2
What is your email address? bruna.marques.moreno@usp.br
What is your assignment token? no
Grade submission failed.
Press ESC if you want to exit this lesson and you
want to try to submit your grade at a later time.

| You almost had it, but not quite. Try again.

|

1: Yes
```

2: No

Seleção: 2

| Keep up the great work!

| You've reached the end of this lesson! Returning to the main menu...

| Please choose a course, or type 0 to exit swirl.

1: R Programming

2: Take me to the swirl course repository!

Seleção: 2

| OK. I'm opening the swirl courses web page in your browser.

| Leaving swirl now. Type `swirl()` to resume.

> `swirl()`

| Welcome to swirl! Please sign in. If you've been here
| before, use the same name as you did then. If you are new,
| call yourself something unique.

What shall I call you? Bruna

| Please choose a course, or type 0 to exit swirl.

1: R Programming

2: Take me to the swirl course repository!

Seleção: 2

| OK. I'm opening the swirl courses web page in your browser.

| Leaving swirl now. Type `swirl()` to resume.

> `swirl()`

| Welcome to swirl! Please sign in. If you've been here
| before, use the same name as you did then. If you are new,
| call yourself something unique.

What shall I call you? Bruna

| Please choose a course, or type 0 to exit swirl.

1: R Programming

2: Take me to the swirl course repository!

```

Seleção:
Insira um item do menu, ou 0 para sair
Seleção: main
Insira um item do menu, ou 0 para sair
Seleção: main()
Insira um item do menu, ou 0 para sair
Seleção: 0

| Leaving swirl now. Type swirl() to resume.

> swirl()

| Welcome to swirl! Please sign in. If you've been here
| before, use the same name as you did then. If you are new,
| call yourself something unique.

What shall I call you? Bruna

| Please choose a course, or type 0 to exit swirl.

1: R Programming
2: Take me to the swirl course repository!

Seleção: 2

| OK. I'm opening the swirl courses web page in your browser.

| Leaving swirl now. Type swirl() to resume.

> bye()

| Leaving swirl now. Type swirl() to resume.
> swirl()

| Welcome to swirl! Please sign in. If you've been here
| before, use the same name as you did then. If you are new,
| call yourself something unique.

What shall I call you? Bruna

| Please choose a course, or type 0 to exit swirl.

1: R Programming
2: Take me to the swirl course repository!

Seleção: 0

| Leaving swirl now. Type swirl() to resume.

> install_course(Exploratory_Data_Analysis)
Erro: objeto 'Exploratory_Data_Analysis' não encontrado

```

```

> install_course("Exploratory_Data_Analysis")
|=====| 100%

| Course installed successfully!
> bye()

| Leaving swirl now. Type swirl() to resume.
> swirl()

| Welcome to swirl! Please sign in. If you've been here
| before, use the same name as you did then. If you are new,
| call yourself something unique.

What shall I call you? Bruna

| Please choose a course, or type 0 to exit swirl.

1: Exploratory Data Analysis
2: R Programming
3: Take me to the swirl course repository!

Seleção: 1

| Please choose a lesson, or type 0 to return to course menu.

1: Principles of Analytic Graphs
2: Exploratory Graphs
3: Graphics Devices in R
4: Plotting Systems
5: Base Plotting System
6: Lattice Plotting System
7: Working with Colors
8: GGPlot2 Part1
9: GGPlot2 Part2
10: GGPlot2 Extras
11: Hierarchical Clustering
12: K Means Clustering
13: Dimension Reduction
14: Clustering Example
15: CaseStudy

Seleção: 8

| Attempting to load lesson dependencies...

| Package 'ggplot2' loaded correctly!

|                                     | 0%

| GGPlot2_Part1. (Slides for this and other Data Science

```

```
| courses may be found at github
| https://github.com/DataScienceSpecialization/courses/. If
| you care to use them, they must be downloaded as a zip file
| and viewed locally. This lesson corresponds to
| 04_ExploratoryAnalysis/ggplot2.)
```

...

```
|= | 2%
| In another lesson, we gave you an overview of the three
| plotting systems in R. In this lesson we'll focus on the
| third and newest plotting system in R, ggplot2. As we did
| with the other two systems, we'll focus on creating
| graphics on the screen device rather than another graphics
| device.
```

...

```
|== | 5%
| The ggplot2 package is an add-on package available from
| CRAN via install.packages(). (Don't worry, we've installed
| it for you already.) It is an implementation of The Grammar
| of Graphics, an abstract concept (as well as book) authored
| and invented by Leland Wilkinson and implemented by Hadley
| Wickham while he was a graduate student at Iowa State. The
| web site http://ggplot2.org provides complete
| documentation.
```

...

```
|=== | 7%
| A grammar of graphics represents an abstraction of
| graphics, that is, a theory of graphics which
| conceptualizes basic pieces from which you can build new
| graphics and graphical objects. The goal of the grammar is
| to "Shorten the distance from mind to page". From Hadley
| Wickham's book we learn that
```

...

```
|==== | 10%
| The ggplot2 package "is composed of a set of independent
| components that can be composed in many different ways. ...
| you can create new graphics that are precisely tailored for
| your problem." These components include aesthetics which
| are attributes such as colour, shape, and size, and
| geometric objects or geoms such as points, lines, and bars.
```

...

```
|===== | 12%
```

| Before we delve into details, let's review the other 2
| plotting systems.

...

|=====| 15%
| Recall what you know about R's base plotting system. Which
| of the following does NOT apply to it?

- 1: Can easily go back once the plot has started (e.g., to adjust margins or correct a typ
- 2: It is convenient and mirrors how we think of building plots and analyzing data
- 3: Start with plot (or similar) function
- 4: Use annotation functions to add/modify (text, lines, points, axis)

Seleção: 1

| That's a job well done!

|=====| 17%
| Recall what you know about R's lattice plotting system.
| Which of the following does NOT apply to it?

- 1: Margins and spacing are set automatically because entire plot is specified at once
- 2: Most useful for conditioning types of plots and putting many panels on one plot
- 3: Plots are created with a single function call (xyplot, bwplot, etc.)
- 4: Can always add to the plot once it is created

Seleção: 3

| One more time. You can do it!

| Which choice is the only one which is inconsistent with the
| other three?

- 1: Margins and spacing are set automatically because entire plot is specified at once
- 2: Plots are created with a single function call (xyplot, bwplot, etc.)
- 3: Most useful for conditioning types of plots and putting many panels on one plot
- 4: Can always add to the plot once it is created

Seleção: 1

| You're close...I can feel it! Try it again.

| Which choice is the only one which is inconsistent with the
| other three?

- 1: Most useful for conditioning types of plots and putting many panels on one plot
- 2: Plots are created with a single function call (xyplot, bwplot, etc.)
- 3: Can always add to the plot once it is created
- 4: Margins and spacing are set automatically because entire plot is specified at once

Seleção: 4

| Not quite, but you're learning! Try again.

| Which choice is the only one which is inconsistent with the
| other three?

- 1: Margins and spacing are set automatically because entire plot is specified at once
- 2: Most useful for conditioning types of plots and putting many panels on one plot
- 3: Plots are created with a single function call (xyplot, bwplot, etc.)
- 4: Can always add to the plot once it is created

Seleção: 1

| Nice try, but that's not exactly what I was hoping for. Try
| again.

| Which choice is the only one which is inconsistent with the
| other three?

- 1: Margins and spacing are set automatically because entire plot is specified at once
- 2: Can always add to the plot once it is created
- 3: Most useful for conditioning types of plots and putting many panels on one plot
- 4: Plots are created with a single function call (xyplot, bwplot, etc.)

Seleção: bye()

Insira um item do menu, ou 0 para sair

Seleção: 0

| One more time. You can do it!

| Which choice is the only one which is inconsistent with the
| other three?

- 1: Most useful for conditioning types of plots and putting many panels on one plot
- 2: Plots are created with a single function call (xyplot, bwplot, etc.)
- 3: Margins and spacing are set automatically because entire plot is specified at once
- 4: Can always add to the plot once it is created

Seleção:

| Leaving swirl now. Type swirl() to resume.